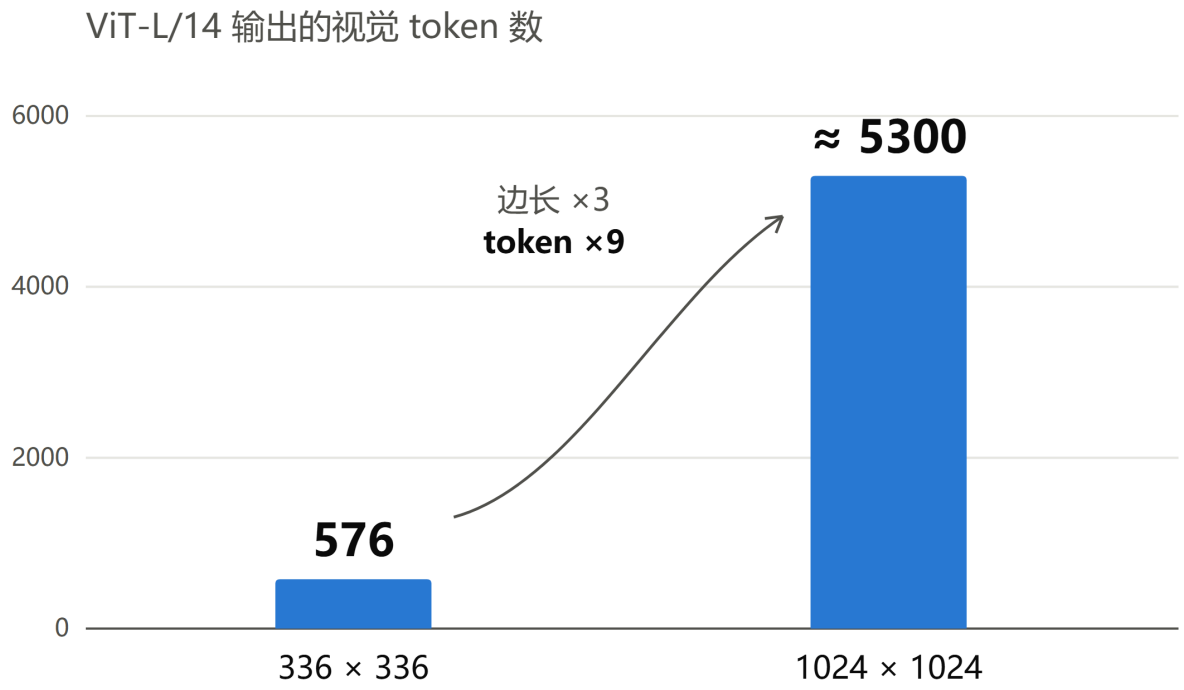
FastVLM

为高分辨率重新设计视觉编码器

CVPR 2025 · Apple

分辨率越高，VLM 越慢

TTFT = 视觉编码时间 + LLM prefill 时间

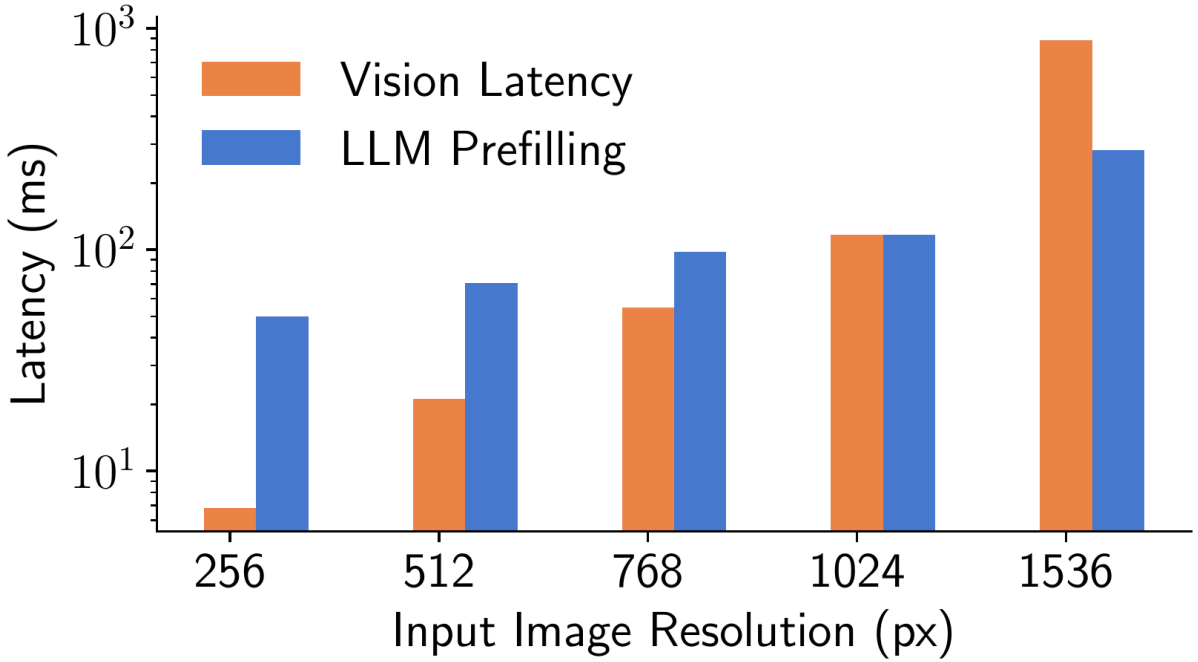


token 数随分辨率**平方**增长

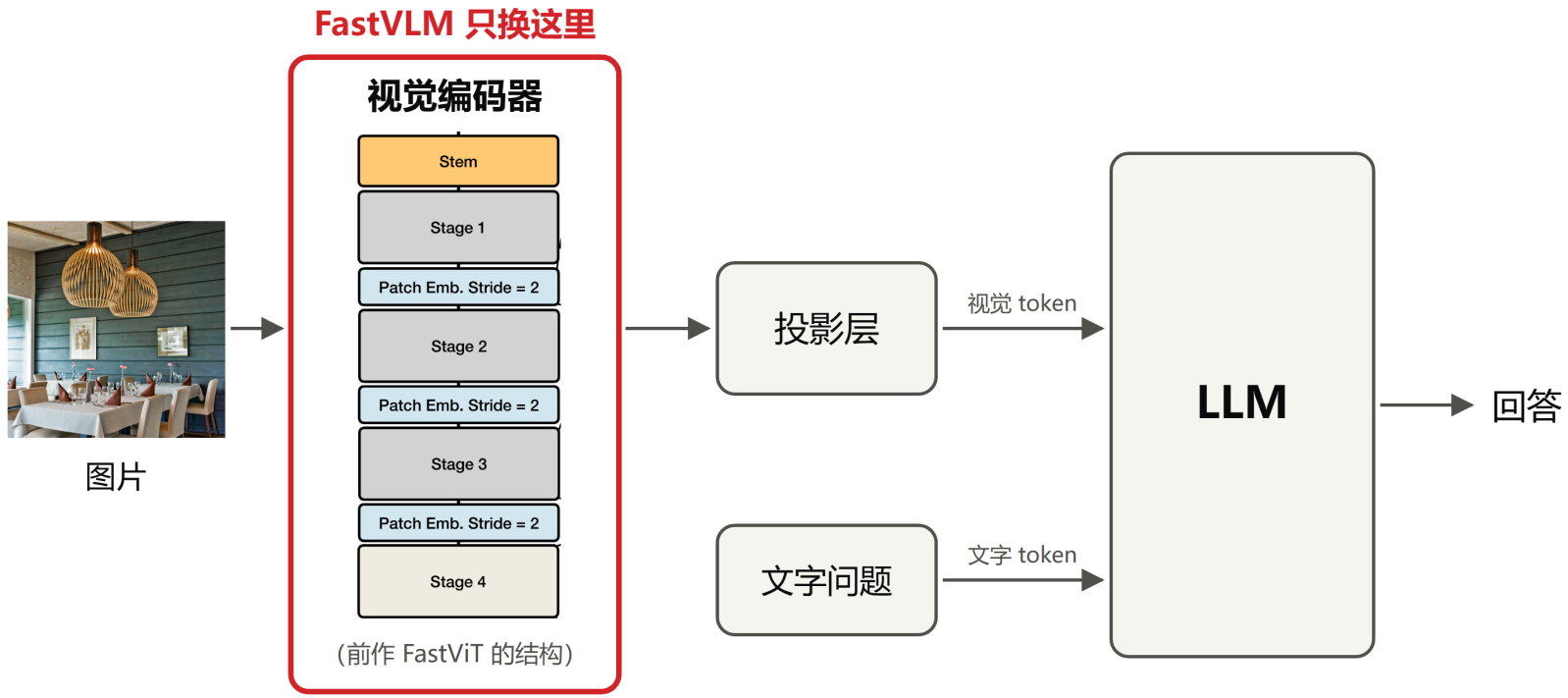
光减少 token 还不够

- 分辨率一高，**视觉编码本身**就是大头
- 剪枝、切图：都在 ViT 外面打补丁

→ 能不能换一个**天生适合高分辨率**的编码器?

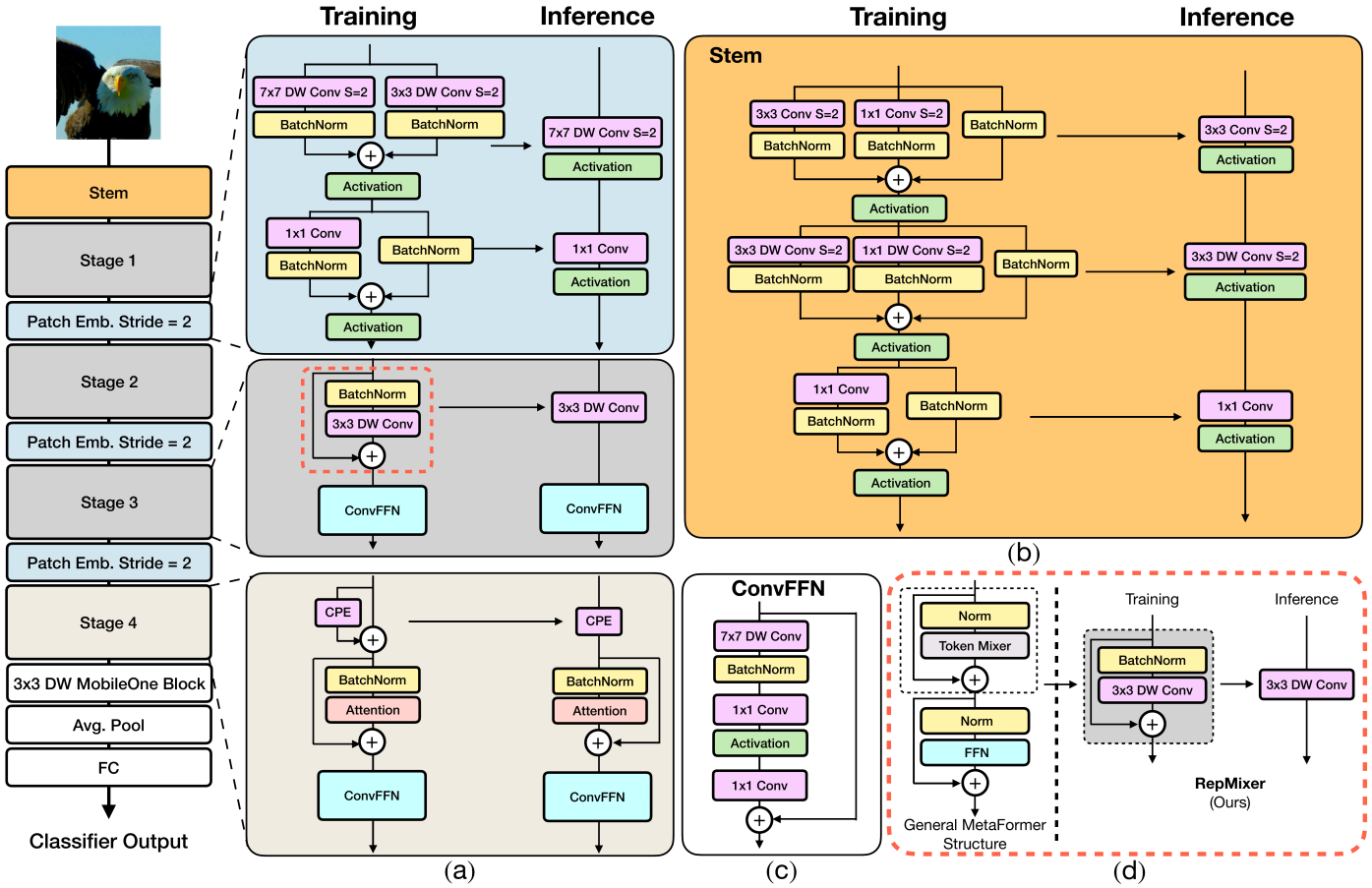


FastVLM 只换了流水线里的一个框



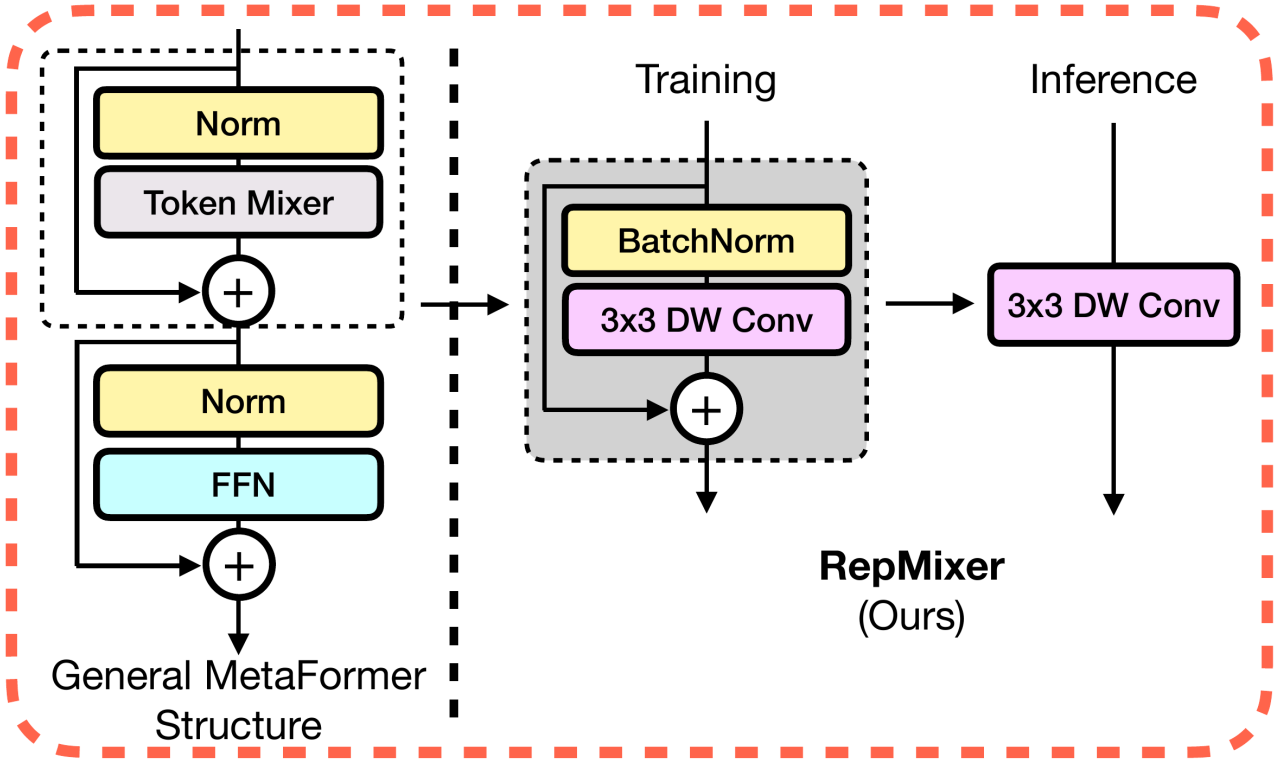
- 只换视觉编码器，投影层和 LLM 沿用 LLaVA
- 后面讲的 stage，都是这个框**里面**的结构

前作 FastViT：整体框架



左边是骨架：4 个 stage 的金字塔 | 右边是零件：每种模块的内部结构

卷积部分没问题：推理时只剩一个卷积



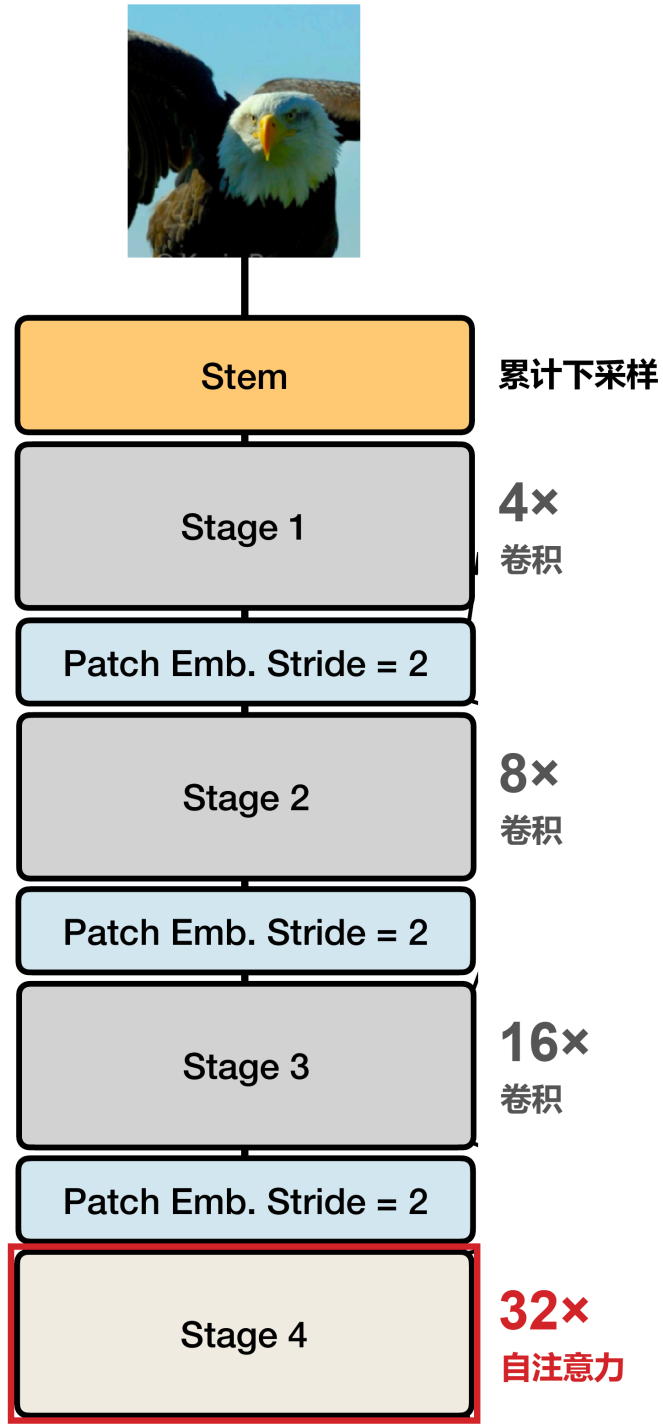
训练时多条支路（好训练）→ 推理时合成一个卷积（跑得快）

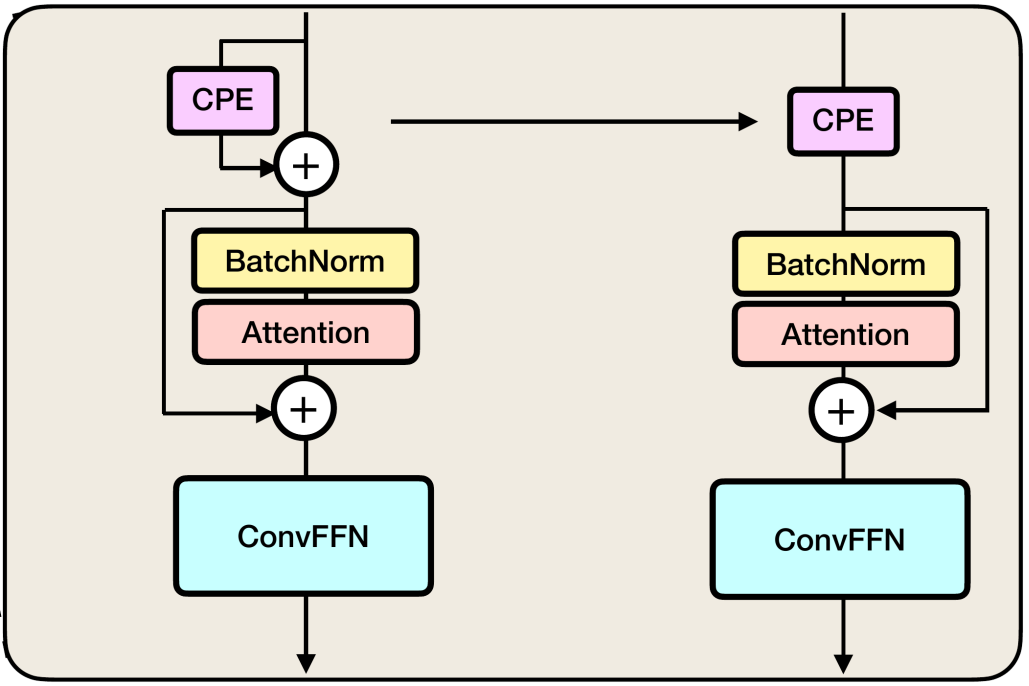
图越大，省得越多 → **这部分适合高分辨率，原样保留**

压了 32 倍，为什么还不够?

- 每过一个 stage 缩小一次，**最后剩几个位置，就是几个 token**
- 1024^2 的图 → 32×32 = **1024** 个 token，交给 LLM 还是太多

→ 能不能**再压**?



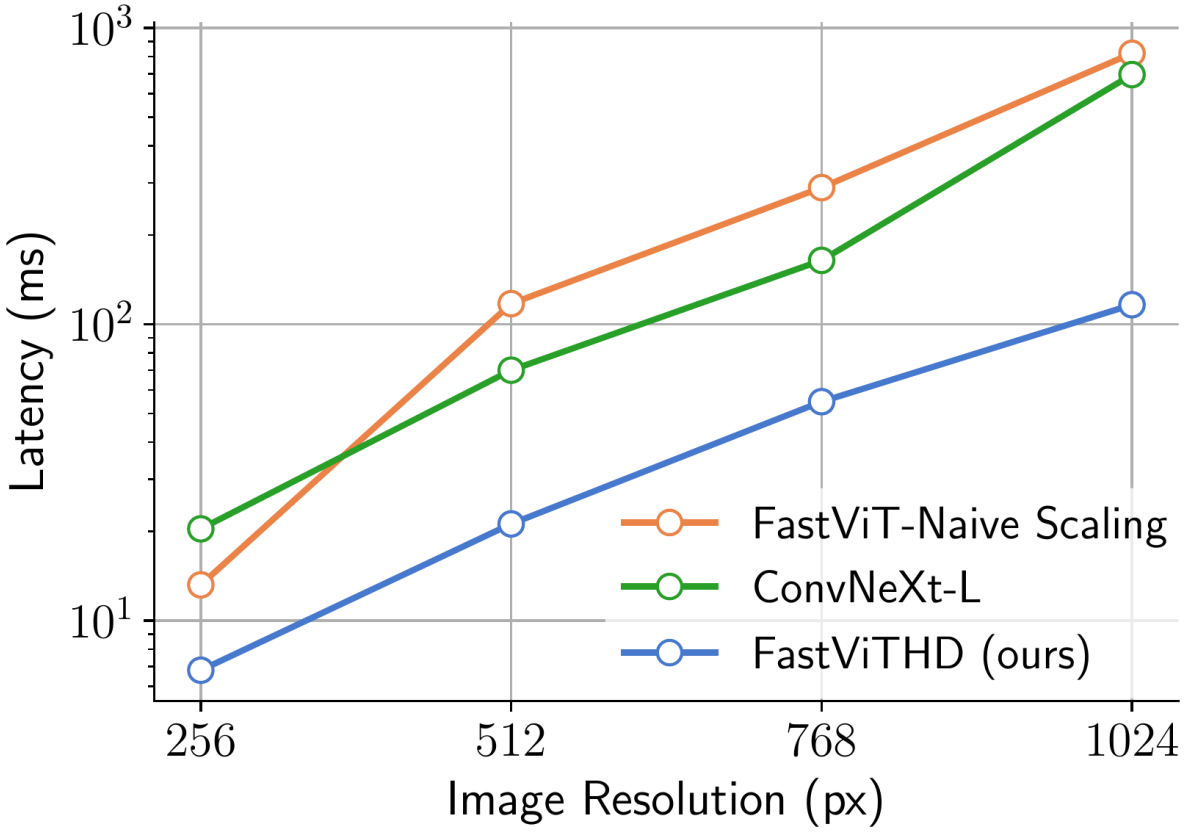


自注意力放得太早

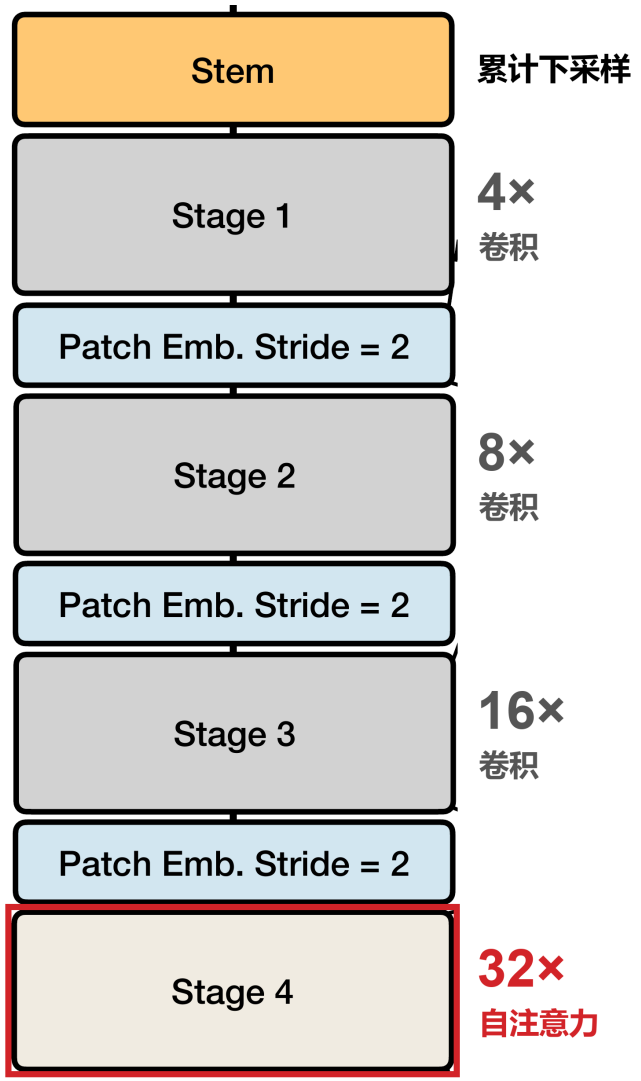
模型放大时，习惯在 stage 3、4 多堆自注意力

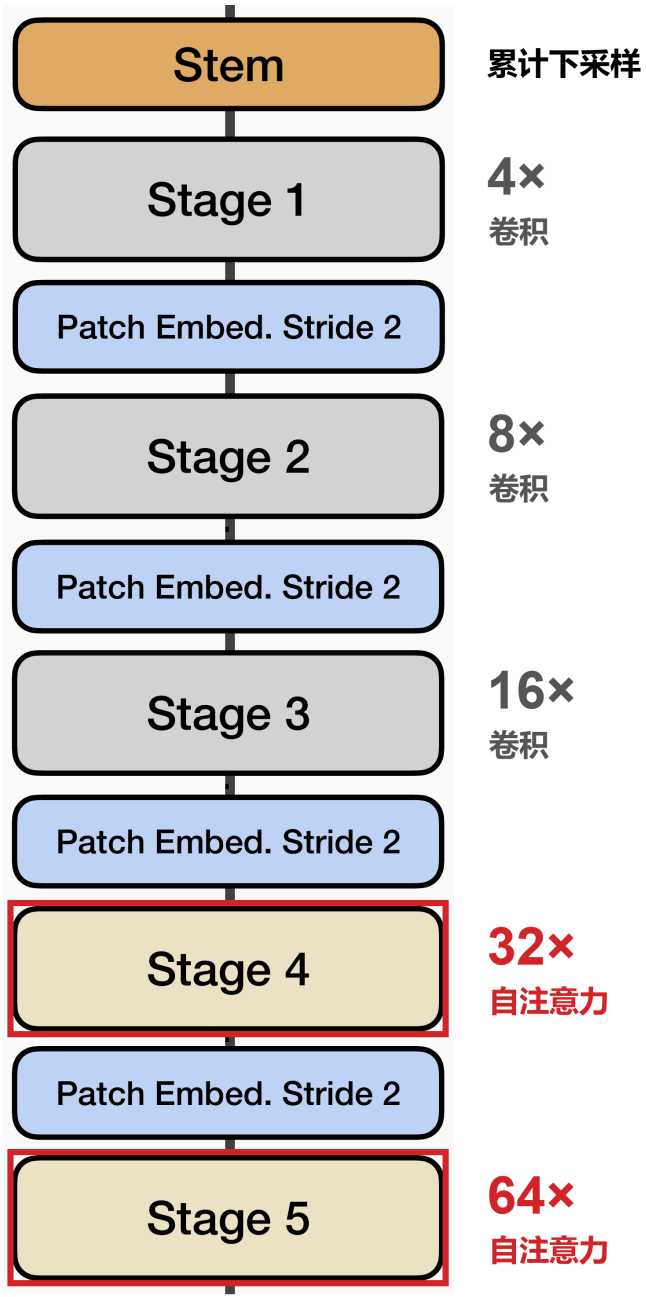
stage 3（压 16 倍）	位置数	自注意力开销
224^2 小图	196	便宜
1024^2 大图	4096	贵约 **400 倍**

实际跑起来：照常规放大，比纯卷积还慢



橙：照常规放大的 FastViT ｜ 绿：纯卷积 ConvNeXt-L ｜ 蓝：FastViTHD（下一页讲）





再压一层

- 加第 5 个 stage，总共压 **64 倍**
- 自注意力挪到 32×、64× 处

ViT	FastViT	FastViTHD
≈ 5300	1024	256

（1024^2 输入的 token 数）

为什么别人没早这么做?

- 压得越狠，一个 token 盖住的区域越大 → **小字容易丢**
- 当时的主流看法：ViTamin 建议总共只压 16 倍

→ 真正的问题：**怎么压，才能不丢东西?**

压 32 倍：一格 = 32×32 像素

品类	6 月销量	环比
饮品	1,284	+6.2%
甜点	963	−3.1%
主食	2,047	+1.8%
小吃	715	+12.4%

注：环比为与上月相比的变化；6 月数据含促销活动期间线上订单 312 件。
本表为演示用的虚构数据。

压 64 倍：一格 = 64×64 像素

品类	6 月销量	环比
饮品	1,284	+6.2%
甜点	963	−3.1%
主食	2,047	+1.8%
小吃	715	+12.4%

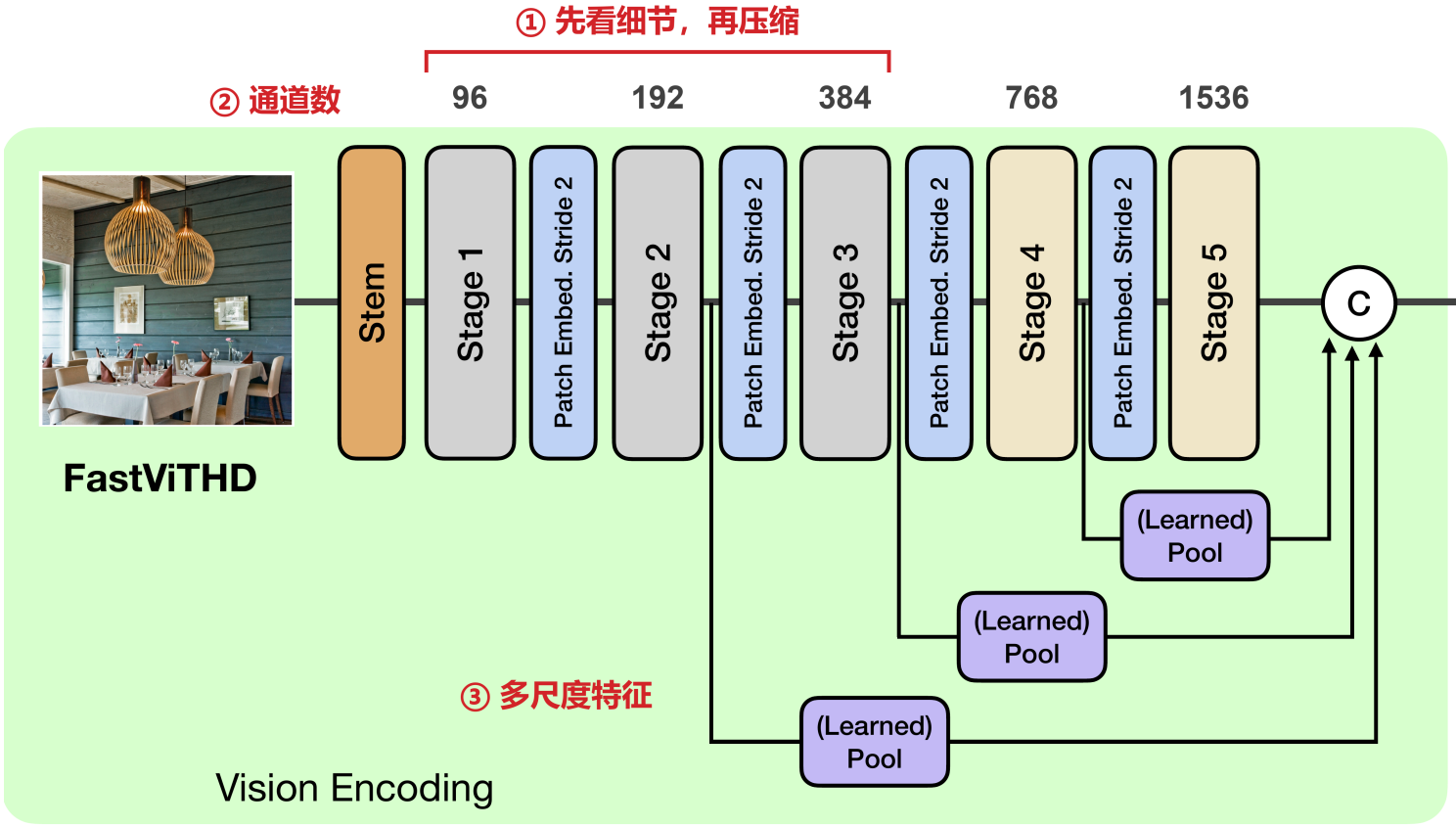
注：环比为与上月相比的变化；6 月数据含促销活动期间线上订单 312 件。
本表为演示用的虚构数据。

怎么压到点上

① 先看细节，再压缩
图大时用便宜的卷积（RepMixer），压小了才做自注意力

② 空间换通道（下一页）

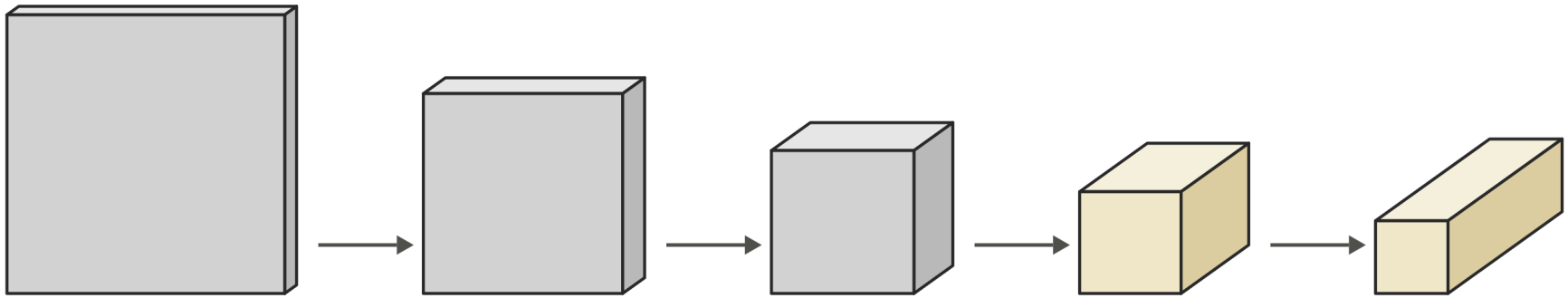
③ 多尺度特征（再下一页）



② 空间换通道：什么是“通道”？

- 特征图 = 高 × 宽 × **通道**
- 一个卷积核卷出一个通道，几个核就有几个通道 → 每个位置有**几种特征**

正面大小 = 长 × 宽（位置数） 厚度 = 通道数

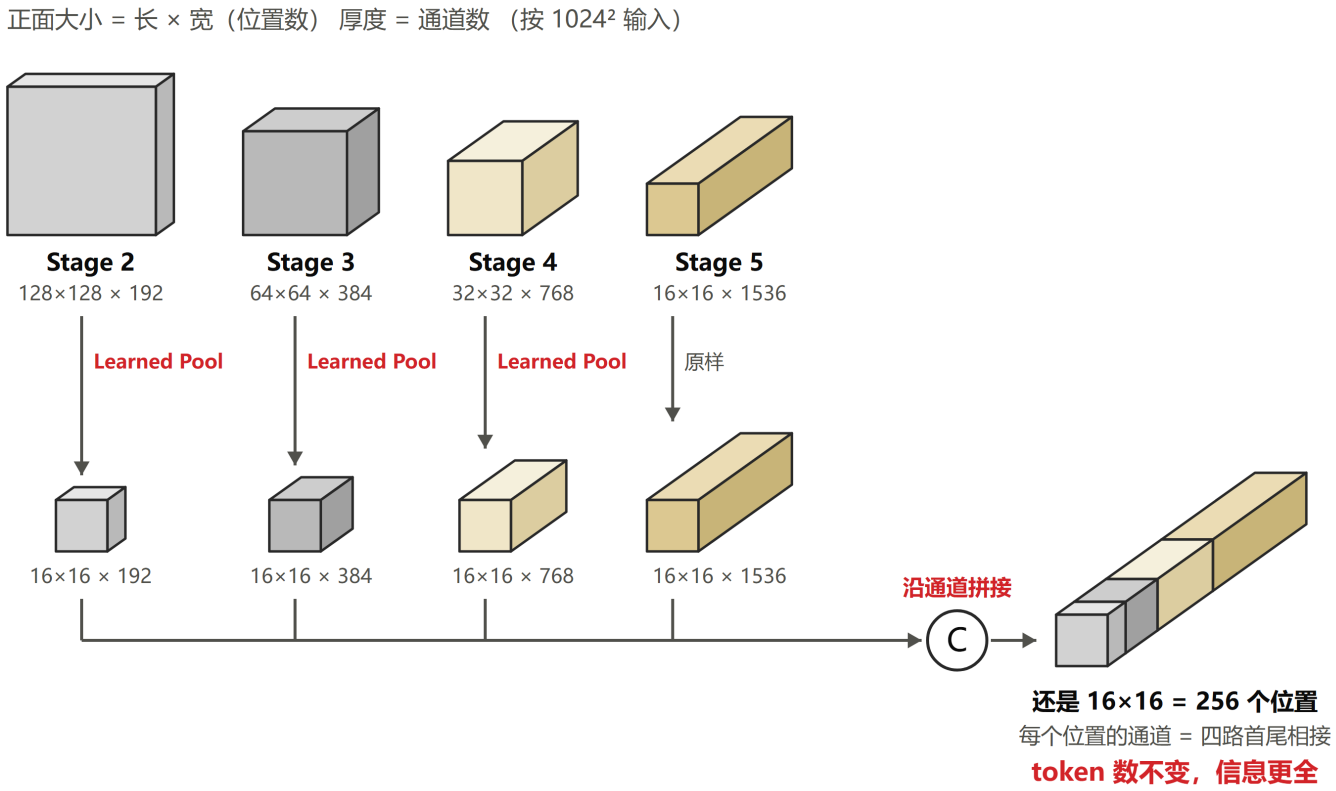


每压一次：长宽减半，通道翻倍 → 每个位置装得更多

③ 多尺度特征：图里的 C 是什么？

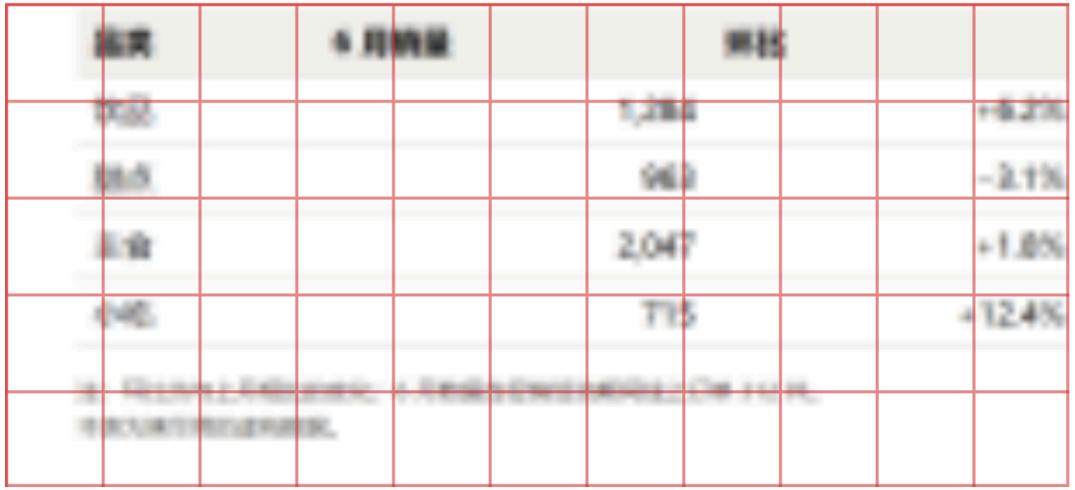
1. 从 stage 2、3、4 各引出一路特征
2. **Learned Pool**：用带步长的深度卷积，缩到和 stage 5 一样大（16×16）
3. **C**：把几路特征沿**通道方向**拼起来

→ token 数还是 **256**，但每个 token 同时带着粗、细两种信息



压缩换来的是分辨率

ViT：224×224，256 个 token



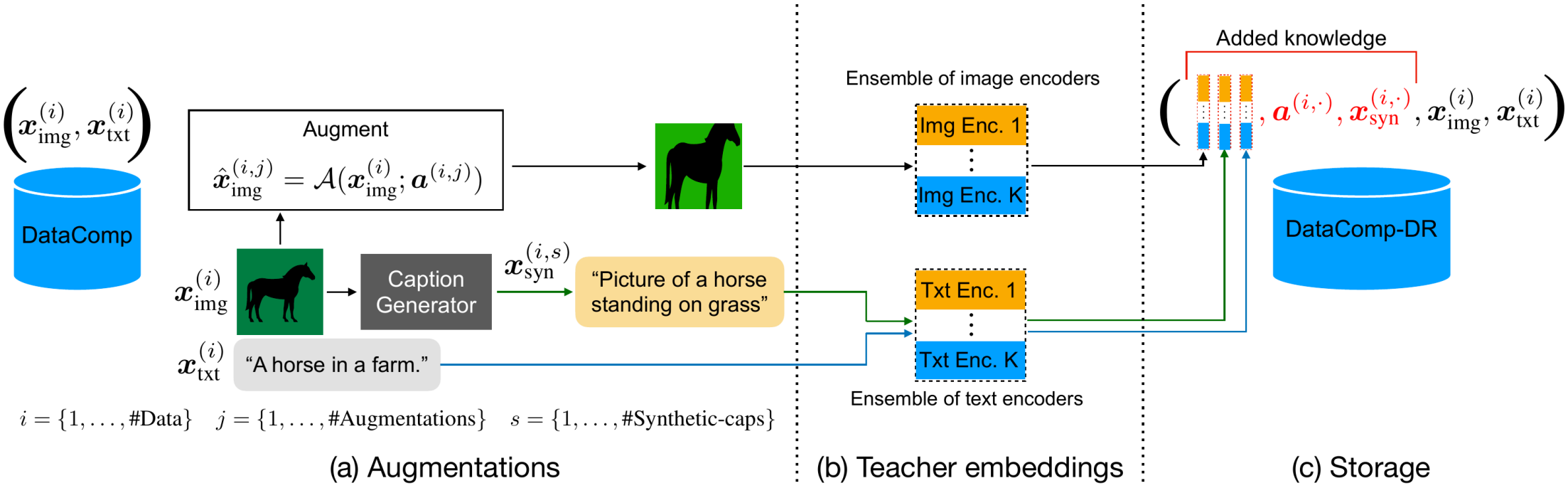
FastViTHD：1024×1024，256 个 token

品类	6 月销量	环比
饮品	1,284	+6.2%
甜点	963	−3.1%
主食	2,047	+1.8%
小吃	715	+12.4%

注：环比为与上月相比的变化；6 月数据含促销活动期间线上订单 312 件。
本表为演示用的虚构数据。

同样 256 个 token，对 VLM 来说，**分辨率才是关键**

代价①：压得狠，就需要更强的训练



每个 token 要装更多信息 → 预训练必须更强

做法来自他们的 MobileCLIP：给每对图文提前备好**更好的配文**和**大模型老师的答案**

代价②：分辨率要和 LLM 大小匹配

- 给很小的 LLM 喂超高分辨率的图，它也**用不上**
- 视觉端压得再好，整个系统还是要两头一起调

从编码器内部重新设计，而不是在外面打补丁